

ProseMapper: A Document Corpus Visualization Project

Evan T.R. Rosenman

August 16, 2012

1 Introduction

ProseMapper is a project designed to visualize a corpus of documents in a user-friendly way. The project encompasses a series of programs which take in a set of text-documents, calculate their pairwise similarities, calculate an optimal spatial layout for nodes representing each document, and visualize these nodes. The node clustering is calculated using LinLogLayout and nodes are represented using Arbor.js. On my personal website, I use slightly different versions of the software to visualize my poetry and articles.

2 Pairwise Document Similarities

Document similarities are calculated using a relatively simple algorithm. Documents are stored in a single folder and read in one at a time, and regular expressions are used to cut up the content of each document into individual words. Documents D are represented as a bag of words with duplicates (“multiset”) in no particular order. Defining $count_{D_n}(i)$ as the function which returns the number of occurrences of term i in document D_n , each document D_n is then converted into a vector W_{D_n} of entries $w_{D_n,i}$ according to:

$$w_{D_n,i} = \frac{count_{D_n}(i)}{\sum_{j \in D_n} count_{D_n}(j)}$$

In other words, $w_{D_n,i}$ represents the frequency of term i in document D_n , excluding stopwords and independent of the order of terms in the document.

The similarity of two documents D_n and D_m is then given by:

$$similarity(D_n, D_m) = \sum_i w_{D_n,i} * w_{D_m,i}$$

A Java program was written to calculate and store the pairwise similarities among all documents in the corpus. However, in initial testing on a corpus of poems, the range of similarity scores between each document and all of its counterparts were shown to vary greatly, meaning that some documents were extremely distinct from all their counterparts while some were very similar to many of their counterparts. Because a graph was desired in which each node had at least one edge, this posed a problem.

A partial solution was found in scaling each node’s similarity scores by its top similarity score. In other words, for each document D_n , we found the largest similarity score $S_{D_n} = \max_m(\text{similarity}(D_n, D_m))$ and redefined each similarity score according to:

$$\text{similarity}(D_n, D_m) = \frac{\sum_i w_{D_m,i} * w_{D_n,i}}{S_{D_n}}$$

This guaranteed that all document similarity scores lay between 0 and 1 and that each document had at least one similarity score equal to 1.0. Notably, however, it meant that our similarity relationship was no longer commutative: $\text{similarity}(D_n, D_m) \neq \text{similarity}(D_m, D_n)$. In order to restore commutativity, we simply took the larger of the two scores involving documents n and m and defined this to be $\text{similarity}(D_n, D_m)$. Lastly, in order to improve the generation of distinct clusters, all document similarities below a certain threshold were automatically set to 0.

3 Clustering and Display

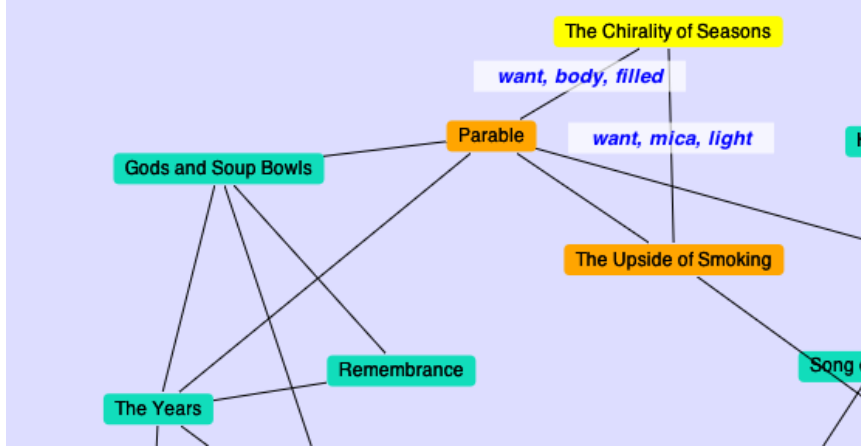
Document similarities were written to a text file, and LinLogLayout was used to generate a graph in which each document (represented as a node) would be optimally displayed in 2-D space. LinLogLayout outputted both the optimal x and y coordinates of each node, as well as a numerical ID corresponding to which cluster (densely connected groups of nodes) the node belonged. All of this information was uploaded to a SQL database.

Arbor.js was then used to generate a graph representing the relationships among documents. LinLogLayout coordinates were converted to Arbor.js coordinates, and a node was generated to represent each document. Next, only those edges above a very high cutoff (0.98) were added to the graph, in order to preserve visual intelligibility.

4 Poem and Article Displays

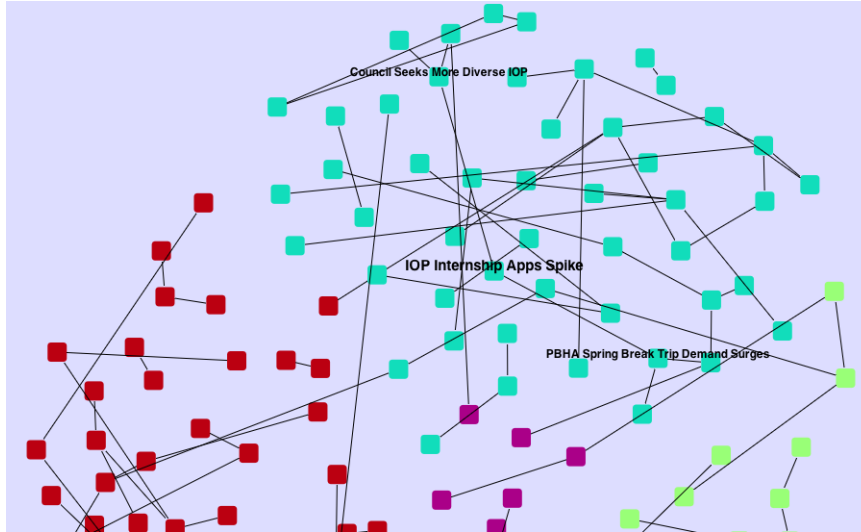
Slightly different methods were used to display the poems and articles on the website. In the case of poems, the cluster designations were discarded. For each

displayed edge, the three words which had contributed the most to the document similarity score (top three terms i such that $w_{D_m,i} * w_{D_n,i}$ are maximal) were stored. Javascript code was written to display these terms over each out-edge of the node currently closest to the cursor. This functionality can be seen below:



HTML and Javascript code was also written to display the content of poems when nodes were clicked.

For the articles display, nodes were colored by cluster, allowing users to visually distinguish the different groups of nodes. Visual inspection revealed that LinLogLayout's clusters corresponded very well to article topicality. Javascript code was written to display an article's name when hovered over, and to also display the name of any connected nodes. This functionality can be seen in the image below:



5 Acknowledgments

Arbor.js and LinLogLayout were indispensable to getting this project running. In addition, Matthew Chartier provided much-needed assistance in figuring out how to integrate the node clustering and node display portions of this project.